

The Client as Interface Specification: API Attack Surface Recovery for AI-Assisted Web Security Assessment

Hasan Flayyih Abdullah

Working manuscript prepared: 1 October 2026

Status: Methodology paper with an accompanying experience report; not peer reviewed

Abstract

Language-model agents work well on software whose source code they can read, because the hypothesis comes from the system itself and can be refuted before anything is sent. Web testing seems to remove that advantage, because the server code is not published and the structure of the interface is not known in advance, so a hypothesis must come from outside the system rather than from it. We argue that the advantage is not removed but moved, and we identify where. A web application's server can only be reached through its interface, and the client that calls that interface is delivered in full to every browser, because the application cannot work otherwise. The client is therefore a recoverable specification of the interface: paths, methods, parameter names, request-body schemas, authentication headers, identifier formats, and often a generated API client or a route manifest. Reading source gives you the implementation; recovering the client gives you the specification. We do not claim these are equivalent. We make the narrower claim that the vulnerability classes ranked highest by the OWASP API Security Top 10 (2023), and their matching CWE entries, are defined by what a server accepts from a caller rather than by how it computes — so for those classes the specification is enough, both to form a hypothesis and to confirm it. Our main contribution is a consequence for measurement: the specification is what makes a request well-formed, and a malformed request looks exactly like an enforced authorization boundary, so a procedure that recovers paths without recovering identifier formats and body schemas produces negative results that are void rather than informative, and cannot tell that this has happened. We set out the resulting procedure, the five controls without which it produces high-confidence false positives, and what it recorded across 45 authorized production targets. We also evaluate the extractor itself against a published OpenAPI specification: 83 per cent of a 294-path interface appears verbatim in the distributed client, supporting the structural premise, while the heuristic that nominates those strings as attack surface proves 32.9 per cent precise over the set as emitted and 9.5 per cent complete — so on that application the weak link is classification rather than extraction, and the controls are necessary rather than advisory. Acting on that diagnosis, we replace the keyword heuristic with call-site evidence and evaluate both rules on three applications, two of them held out. One result generalizes: the repaired rule recovers 100 per cent of the extraction ceiling on all three, which is the effect of no longer rejecting rather than of classifying better. Two do not. Precision is application-dependent, falling 13.3 points on one client and rising 12.4 on another, and the per-route evidence tier separates precision by 40.1 points on one application and 9.2 on the second, against a 20-point bar, while the third admits no assessment of it, so the tier is not a general operating point. The ceilings themselves — 98.8, 62.2 and 82.8 per cent — show extraction to be the larger loss wherever a client builds paths at runtime. We make no claim about how any programme judged those outputs.

Keywords: AI-assisted security assessment; web application security; API attack surface enumeration; client-side artefact analysis; authorization testing; broken object level authorization; research methodology

1. Introduction

Recent results show that language-model agents can find and exploit vulnerabilities in software whose source code they are allowed to read [1,2]. That setting supplies the property that makes human code review work: the implementation is visible, so a hypothesis can be formed from the code and refuted by reading further, before

anything is executed. The hypothesis comes from the system under test, and refuting it is cheap.

Web testing appears to remove that property. The server code is not published. What is visible is a rendered page and whatever requests a given session happens to send. Where the structure of the interface is not known in advance, requests must go out before it is known, and the hypothesis then comes from outside the system rather than from it.

This paper does not argue that one discovery method yields more than another, and it reports no comparison of yield. Its subject is what a recovered interface specification entitles an assessor to conclude, and under what conditions a result measured against it is valid rather than void. We argue that the property above is not removed but moved, and that locating it correctly changes what AI-assisted web testing can establish. The argument rests on how modern web applications are built, not on any capability of a particular model: the server can only be reached through the application's interface, and the client that calls that interface is delivered in full to every browser. Whatever the client must encode to build a well-formed request is therefore available by reading it.

Section 2 states what each setting makes visible, and declines the claim that the two are equivalent. Section 3 gives three arguments that the client is a sufficient source for a specific and consequential set of vulnerability classes; Section 3.3 is the paper's main contribution, and it concerns the validity of measurement rather than the yield of discovery. Section 4 sets out the procedure and Section 5 the controls it requires. Section 6 reports what the procedure recorded; Section 6.1 reports an unmatched dictionary-enumeration pilot and explains why it is not a control arm; and Section 6.2 evaluates the extractor itself against a published specification, reports a poor result under a commitment recorded before the measurement, works through what that result implies for the figures in Section 6, then repairs the classifier the measurement indicted and evaluates both rules against three applications, two of them held out, under one pre-registered criterion. Sections 7 and 8 cover threats to validity and prior work, where endpoint recovery from client code is already established and this paper's narrower contribution is positioned against it.

2. Observability in the two settings

A methodology paper that overstates its epistemic position is easy to refute. The comparison therefore begins by conceding what client recovery does not supply.

Source inspection renders the implementation observable. The assessor observes the authorization predicate itself, data flow from entry point to security-relevant sink, parsing and encoding decisions, trust assumptions encoded in control flow, and internal services exposed through no public interface. A defect whose nature is computational — untrusted input reaching a query constructor, unsafe deserialization, an invariant violation internal to a transaction — is observable by reading.

Client recovery renders the interface specification observable. The assessor recovers which routes exist, which methods each accepts, the schema of each request body, the field identifiers the client transmits, the identifier encoding each route expects, the headers conveying authentication state, and frequently the application's complete route table. None of this renders the server-side computation observable. The authorization predicate is not observable; only its effects are.

These constitute distinct categories of knowledge, and the paper does not equate them. The proposition advanced below is narrower, and its narrowness is the condition of its defensibility.

3. Sufficiency of the interface specification for the highest-ranked vulnerability classes

3.1 The client is the invoking party and is distributed in its entirety

A web application accepts input exclusively through its interface. Every request the application itself issues is therefore constructed within client code, and that client is distributed to the assessor as a necessary condition of the application functioning. In client-rendered architectures this is concrete: the deployed bundle contains the request-construction logic, commonly in the form of a generated API client, and typically a route manifest enumerating views together with the resources each retrieves.

This is a structural property rather than an incidental disclosure remediable by improved minification. Code that must construct a request must encode what the request requires. Identifier minification renames bindings; it does not eliminate the literal path, the method, or the field identifiers the server will parse. The property is a necessary precondition of the argument, and Section 7 states the architectures in which it does not hold.

3.2 The vulnerability classes determined by the interface

The classes the standard interface taxonomy ranks highest are determined by what a server accepts from a caller rather than by how it computes a result. The ranking is not this paper's assertion: the OWASP API Security Top 10 (2023) places broken object level authorization first, broken object property level authorization third, and broken function level authorization fifth [3]. Stated against that taxonomy and the corresponding CWE entries [4]:

Class	Taxonomy	How the test is run
Broken object level authorization	API1:2023; CWE-639, <i>Authorization Bypass Through User-Controlled Key</i>	Transmit a reference to a non-owned object as a subject lacking entitlement
Broken object property level authorization, including mass assignment	API3:2023; CWE-915, <i>Improperly Controlled Modification of Dynamically-Determined Object Attributes</i>	Inject a field the client never transmits, then retrieve the object as a second subject
Broken function level authorization	API5:2023	Invoke a privileged route as a subject holding a lesser role
Unrestricted access to sensitive business flows	API6:2023	Invoke a flow outside the sequence the interface imposes
Workflow and state-transition enforcement failure	CWE-841, <i>Improper Enforcement of Behavioral Workflow</i>	Execute a transition from a state that should preclude it
Absent server-side enforcement of a client-side constraint	CWE-602, <i>Client-Side Enforcement of Server-Side Security</i>	Transmit a request violating a constraint the client enforces
Undocumented or unmanaged interface inventory	API9:2023, <i>Improper Inventory Management</i>	Enumerate routes absent from documentation and observed traffic

The rows are not mutually exclusive, and the paper does not treat them as a partition: API6:2023 and CWE-841 overlap wherever a sensitive flow is also a state machine, and CWE-639 and API1:2023 describe the same defect at different levels of abstraction. The table enumerates test operationalizations, not disjoint categories.

For each entry, the confirming evidence is a request together with a contrast across subjects, and no element of that evidence requires the implementation. Establishing that one tenant retrieves another tenant's records requires a well-formed request and two subjects; source inspection would explain *why* the predicate is absent, but is not required to establish *that* it is absent. This delimits the claim: the interface specification is sufficient here and insufficient for injection and memory-safety classes, where the defect is computational.

API9:2023 deserves separate mention, since it is the condition the recovery procedure directly addresses: an interface inventory that is incomplete with respect to the deployed surface. Recovery from client artefacts is an

enumeration method for precisely the surface this class describes.

3.3 The specification determines request well-formedness, and a malformed request is not distinguishable from an enforced boundary

This proposition concerns measurement validity rather than discovery, and is the strongest ground for inspecting the client.

A test constitutes evidence only if its legitimate instantiation succeeds — the positive control condition formalized in Section 5. Where a route receives an identifier in an incorrect encoding, it refuses the request, including when the requesting subject is the rightful owner. The response is observationally equivalent to a correctly enforced authorization boundary. The assessor records a negative result, and the route was never in fact exercised.

Instances of this failure recur and are not atypical. Identifiers belonging to one resource type, supplied to a route expecting another, produce a uniform refusal across all subjects. A route requiring a type-prefixed identifier may be adjacent to sibling routes requiring the unprefixed numeric form, because the invoking code strips the prefix for one and not the others. A uniform refusal across both the owner and the non-owner arm presents as a permission predicate and in fact constitutes a malformed request on both arms.

Neither dictionary enumeration nor crawling resolves this, as both operate on the syntactic form of a path rather than on the contract of its invoking code. Inspection of the invoking code resolves it directly, the encoding being present in the construction logic. The operational rule follows: **where the owner arm is refused, the identifier encoding is to be suspected before any inference is drawn concerning the route.**

The same reasoning governs request bodies. Knowledge of the fields the client transmits is the precondition for injecting a field it does not transmit, which constitutes the entirety of the API3:2023 test. Absent the schema, there is no principled basis for selecting an injection candidate.

A worked instance from the corpus of Section 6 illustrates the dependency, stated without identifying the programme. A registration endpoint on a multi-tenant storefront platform accepted a boolean field governing whether the newly created account required email activation. The field was constructed by the client, transmitted on every registration, and honoured by the server: setting it to false produced an account in an activated state without the applicant demonstrating control of the address. The defect instantiates CWE-602, *Client-Side Enforcement of Server-Side Security*, and its proof is a single request. It is, however, unreachable by any procedure that does not possess the body schema, because the test consists of altering one named field among several and no enumeration strategy nominates that field. The field name is recoverable only from the invoking code. The same defect was subsequently confirmed on three distinct assets served by the same platform, which is reported in Section 6 as one defect rather than three findings.

3.4 Equivalence of inferential structure with source inspection

Property	Source inspection	Client recovery	Dictionary enumeration
Hypothesis comes from	the system's own source	the system's own client	an external wordlist
Falsifiable prior to transmission	yes	yes	no
Confirmation procedure	execution or test	contrasted request across subjects	response status
Informativeness of a negative	yes, scoped to inspected code	yes, scoped to recovered specification	limited to enumerated candidates

Source inspection and client recovery share an inferential structure: inspect the system's own artefacts, derive a hypothesis the artefacts render falsifiable, then confirm empirically. Dictionary enumeration has a different structure, and the final row follows from that difference by definition rather than by measurement: a negative result for a path never enumerated is not a statement about the application. The table classifies inferential structure, not relative yield: which method surfaces more routes in practice is an empirical question whose answer would not bear on the validity argument this paper makes. The claim advanced is that client recovery occupies the first structure, not that it achieves the observability of source inspection.

A corollary follows. Client recovery and traffic observation are not substitutable. An observed session exposes only the code paths that session exercised, and deferred module loading entails that a given interaction session retrieves a fraction of the deployed client code. The corpus of Section 6 bears on the magnitude of this effect: across the targets for which both sources were collected, their route sets were very nearly disjoint. A procedure employing only one therefore carries a coverage gap that is substantial and, from within that procedure, unobservable.

4. Specification of the assessment procedure

Each stage is motivated by a determinate failure observed on its omission.

Stage 1 — Edge characterization, per host. Probe each host with a valid path and a non-existent control path, under multiple methods, comparing response bodies rather than status codes. This establishes three facts on which all subsequent inference depends: whether status constitutes the result, whether the host responds uniformly, and whether a command-line client reaches the origin or an instrumented browser is required. A single characterization generalized across an estate is incorrect for at least one host in practice.

Stage 2 — Subject establishment and positive controls. Authorization propositions require a minimum of two subjects, verified distinct and verified live prior to testing. For each subject, establish a request returning substantive data and retain it as that subject's positive control.

Stage 3 — Client artefact recovery, requiring no session. Deployed bundles are public artefacts. The unauthenticated entry document, present in any application exposing a login, serves the runtime, the entry bundle, and in most cases the route manifest. Recover deferred modules by parsing the module map; where the map encoding does not conform to a recognized form, inspect the URL-construction function directly. An empty recovery constitutes a procedural failure, not an observation about the target.

Stage 4 — Integration of observed traffic. A capture of genuine product interaction contributes what bundles do not: routes of confirmed reachability with authentic request bodies. Given the corollary in Section 3.4, this stage is not discretionary; bundle recovery and traffic capture each cover surface the other omits.

Stage 5 — Recovery of the specification rather than the path. For each route, recover method, parameter identifiers, body schema, identifier encoding, authentication header, and the base-URL constant resolved from the invoking code. A path whose base constant does not resolve does not constitute surface and is classified as unresolved rather than counted. The invoking code, not the syntactic form of the path, is authoritative for each of these attributes.

Stage 6 — Inventory construction with provenance, and defence of the denominator. Each route receives a record stating its source. Counting extracted string literals as attack surface overstates it: vendor SDK paths, lexical artefacts of embedded tooling, and unresolved bases are all superficially endpoint-shaped. The denominator must be stated and defended prior to any coverage assertion.

Stage 7 — Selection for falsifiability. Self-scoped routes exposing no tamperable identifier are overwhelmingly non-vulnerable and exhaust the request budget. Allocate requests where identifiers occur: path parameters, tenant and account parameters, nested child resources addressable by their own routes, export and reporting functions.

Stage 8 — Data-plane instantiation prior to testing. The isolation proposition concerns whether one subject retrieves another's data, and is not answerable while the second subject's account is unpopulated. Instantiate a small number of inert, uniquely marked records in every subject's account and retrieve them, since acknowledgement of ingestion does not establish persistence.

Stage 9 — Contrast with pre-registered expectation. State the expected outcome per subject per route prior to transmission. An expectation recorded subsequent to the response is rationalized rather than tested.

Stage 10 — Request instrumentation. Record route, method, acting subject, transmitted content, security-relevant response content, status, and the responding tier — edge or application. An attribute the assessor must remember to record is omitted; an attribute the instrumentation records as a consequence of transmission is not. The instrumentation has one structural gap, and it is worth naming here rather than leaving it to be discovered: it is keyed on the request, not on the finding. Each row records what was sent and what returned, but nothing links a finished finding to the specific rows that evidence it, and nothing links an external decision about that finding back to either. Within an assessment this is immaterial, since the rows are resolvable by hand. Across assessments it is the reason Section 6 defers reporting external adjudication: the linkage that would make such a report checkable was never recorded, because no stage requires it. A finding-level provenance key, written at the moment a draft cites a row, would close the gap and is the one amendment to this procedure the present work would recommend.

Stage 11 — Escalation of demonstrated impact, not of induced effect. Initial confirmation establishes that a boundary fails, not what an adversary obtains. Raise demonstrated impact until it cannot be raised with the available subjects, then terminate, and demonstrate reachability rather than exercising it: an enumeration and two retrievals, not bulk extraction.

5. Validity controls

These are not implementation details. Each marks a failure the recovery advantage renders *more* probable, since rapid surface recovery yields rapid candidate generation.

Control 1 — Positive control. A test whose legitimate instantiation also fails has no discriminative power and establishes nothing. Prior to accepting any negative result, confirm that the legitimate path returned substantive data within the same batch. A negative result without a concurrent passing control constitutes absence of measurement rather than evidence.

Control 2 — Tier attribution. Refusal by an intermediary is not refusal by the application. A web application firewall rejection, a bot-management interstitial, and a rate-limit response occupy the same status field as an application decision and carry the opposite meaning. Every inference must record whether the request reached the origin.

Control 3 — Boundary specification. The crossed boundary must be named: subject to subject, tenant to tenant, role to role, unauthenticated to authenticated. Self-scoped effects, public data served over an interface, and client-side-only constraints constitute observations rather than findings. An observation recorded without an attached verdict accrues unwarranted confidence under restatement.

Control 4 — Identifier encoding derived from the invoking code. This is Control 1 under a specific instantiation, argued in Section 3.3. A refusal on the owner arm is evidence concerning the request, not concerning the route.

Control 5 — Acting subject derived from the transmitted credential. An assessor who believes they are acting as the second subject while transmitting the first subject's credential invalidates every negative result measured within that slice, and no other element of the procedure detects the condition. Fingerprint the credential per request and record the derived subject identity rather than the asserted one.

A sixth practice is warranted by what it produces rather than what it prevents: a candidate the procedure invalidates constitutes a complete result. Recording the grounds of invalidation, with its evidence, is the output that preserves the credibility of the surrounding negative results.

6. Experience report

The procedure was applied to 45 authorized targets across 23 bug bounty programs. The figures below enumerate the procedure's own recorded outputs, and nothing in them asserts a judgment by any receiving programme. External adjudication of a subset of these outputs does exist, in the form of vendor-assigned vulnerability identifiers and programme awards; reporting it is deferred to separate work, because doing so credibly requires per-finding provenance linking each external decision to the specific artefacts that produced it, and this corpus does not carry that linkage.

Procedural stage	Recorded output
Authorized targets with retained artefacts	45, across 23 programs
Client artefacts recovered	15,105 files
Routes inventoried with provenance	5,652, across the 30 targets carrying an inventory
Routes recovered from client artefacts but absent from observed traffic	1,375, on the 19 targets carrying both sources
Intersection of the client-recovered and traffic-observed route sets	73, of a 3,805-route union
Requests transmitted and instrumented	7,661, none malformed on parse
Requests attributed to the application tier	7,110
Requests attributed to an intermediary tier	83
Requests whose responding tier was not determinable	468
Distinct acting subjects, derived from transmitted credentials	94 identifiers
Typed lead records	812, across 33 targets
Candidates invalidated by the procedure itself	20
Evidenced negative results recorded	26
Report drafts satisfying the procedure's internal severity gate and adversarial review	13
Distinct defects those drafts represent, after root-cause clustering	11, across 10 targets

Four rows are of principal methodological significance.

Source intersection, and what it does not show. On the nineteen targets carrying both a client-recovered and a traffic-observed route set, the two sets intersect in 73 routes of a 3,805-route union. Client recovery contributed 1,375 routes that traffic observation did not; **traffic observation contributed 2,357 that client recovery did not.** Traffic observation therefore yielded the larger unique contribution, by a wide margin, and the figures are stated in that order deliberately: the claim of this paper is non-substitutability, not client primacy. Nothing in the argument requires the client to be the richer source, and Section 3.1 asserts only that the client is a *recoverable* and *complete* specification of what the application itself invokes. The measurement on which Section 3.4's corollary and Stage 4's

non-discretionary status depend is the near-disjointness, and that is what the intersection of 73 establishes. The figure is a set intersection over the complete inventories of those targets and involves no selection.

Tier attribution. Of 7,661 instrumented requests, 7,110 were attributed to the application tier, 83 to an intermediary, and 468 were not determinable from the record. Control 2 is thereby satisfied across the substantial majority of the corpus: the status codes underlying these figures describe application behaviour rather than a protective layer preceding it. A corpus unable to report this ratio could not support inference of any kind.

Derived subject identity. Ninety-four distinct acting subjects were derived by fingerprinting the credential present on each request rather than by accepting an asserted label. Control 5 is satisfied by construction: a slice transmitting an unintended subject's credential is observable in the record.

Internally invalidated candidates. Twenty candidates were invalidated by the procedure itself, alongside twenty-six separately recorded evidenced negative results. This figure distinguishes a method from an undisciplined candidate-generation procedure. A procedure recovering surface rapidly generates candidates rapidly, and the sole internal evidence that its gate exercises discrimination is a record of candidates the gate arrested. Eleven defects satisfied the gate; twenty candidates did not. One instance is worth stating because it shows the gate operating on the evidence layer rather than on judgment: a completed draft was withdrawn after audit established that its proof of concept asserted a status code and a response header for which no instrumented row existed.

Three qualifications apply. Provenance and lead-type attributes are free-text labels assigned by the procedure's own operators rather than values emitted by a calibrated instrument; no inter-rater reliability measure exists for them, and their categorial consistency is bounded by the consistency of the practice that produced them. The thirteen drafts resolve to eleven distinct defects because one defect was confirmed on three assets of a shared platform and one draft exists in two variants; reporting the draft count alone would treat one root cause as three results, which is the error the clustering column exists to prevent. And the eleven defects satisfied *this procedure's* severity gate and adversarial review, which are internal criteria, as stated at the head of this section.

6.1 Dictionary enumeration in this corpus

Dictionary enumeration was also executed against twelve hosts in this corpus, under soft-404 baselining and with rate-limited responses excluded, yielding 40 confirmed paths in total, thirty of them on a single host. On the one host where a dictionary scan and a client-recovered inventory exist for the same origin, the scan returned no confirmed path while the inventory held sixteen routes.

These figures are reported for completeness and support no comparison. The request budgets were not matched, the wordlists were curated per host rather than held constant, the twelve scanned hosts barely overlap the nineteen carrying both route sources, and scan effort was allocated by assessor judgment rather than by design. No inference about relative method yield should be drawn from them in either direction.

A controlled yield comparison is specifiable — a fixed target set, an identical per-target request budget, one wordlist held constant, the two methods executed independently, and the resulting route sets compared for unique contribution and for confirmed liveness under identical probing — and would be a worthwhile separate study. It is not required by any claim advanced here.

6.2 Extractor accuracy against published specifications, on three applications

Section 8 notes that the regex-based extractors dominating practice carry no published precision or recall evaluation. That criticism applies to the extractor used here, and this subsection addresses it.

Pre-specified reporting commitment, recorded before the measurement was taken. The extractor is evaluated against a published OpenAPI specification serving as ground truth, on an open-source single-page application whose built client is publicly distributed. Precision — the proportion of extracted first-party routes that appear in the

specification — is reported exactly as measured. A low value would not contradict the argument of this paper. It would quantify the limitation Stage 6 already asserts qualitatively, namely that extracted string literals are not automatically attack surface, and it would strengthen rather than weaken the case that the controls of Section 5 are necessary rather than advisory. A high value would support the reliability of the inventory on which Section 6's figures rest. Both outcomes are reported in the same terms.

Recall against the full specification is deliberately not reported as an accuracy figure. A client legitimately invokes only a subset of a published interface, so the specification is not a valid denominator for what the extractor should have found; such a ratio would measure the client's coverage of the API rather than the extractor's fidelity. Where a recall-shaped figure is given below it is labelled as specification coverage and carries no claim about extractor accuracy.

Target and ground truth. The evaluation used Jellyfin, an open-source media server whose maintainers publish an OpenAPI 3.0.4 specification, against the matching release of its single-page web client. The specification declares 294 paths at version 12.1.0; the client is the portable server distribution of the same version, comprising 986 minified and code-split JavaScript bundles totalling roughly 28 MB. Version alignment between specification and client is exact, which removes the most obvious confound in a comparison of this kind. Both artefacts are publicly distributed and the measurement involved no request to any running system.

The structural premise, measured. Of the 294 declared paths, 244 — 83.0 per cent — appear verbatim as literal strings in the distributed client bundles. Section 3.1 argues on structural grounds that the client must encode what a well-formed request requires; this figure is the first direct measurement of that premise offered here, and it supports it. The 50 absent paths are predominantly administrative and streaming-variant routes the web client does not invoke, which is the expected pattern: a client encodes the interface it uses, not the interface that exists.

Extraction and classification fail separately, and on two of the three applications extraction is the larger loss. The Jellyfin measurement separates the two; the three-application figures below locate each. After normalizing parameter placeholders on both sides, 243 distinct specification paths are present in the client. The extractor reported 240 of them, or 98.8 per cent, somewhere in its output. Of those same 243, however, only 23 — **9.5 per cent** — were admitted to the set the extractor labels API-shaped, which is the set an assessor would treat as attack surface. Expressed against the whole published interface rather than against the subset the client encodes, the same 23 paths are 7.8 per cent of the 293 declared. That second ratio is reported here under the label committed to above, **specification coverage**, and carries no claim about extractor accuracy: a client is not obliged to invoke every route a server offers. Precision on that set is correspondingly poor. The set contains 73 rows, of which 24 appear in the specification: **32.9 per cent**, and that is the headline figure, because it describes the set as the tool actually emits it. It decomposes as follows. Eleven of the 73 rows are absolute third-party URLs — documentation links, a React Router upgrade notice, a Google cast loader, a map search endpoint — every one a false positive. Setting those aside as out of scope for first-party surface leaves 62 relative paths of which 24 match, giving 38.7 per cent; that variant is reported for completeness and is not the figure quoted elsewhere in this paper. The 24 matching rows resolve to 23 distinct normalised paths, because `/Users` and `/Users/` are both emitted; precision is therefore computed over rows and recall over distinct paths, which is why the two figures differ by one. Thirty-five of the thirty-eight false positives fall into one identifiable class — client-side router paths such as `/dashboard/users/:userId/:tab` and `/login`, which are views rather than server endpoints. Excluding that single class raises precision to 24 of 27, or **88.9 per cent**, and leaves only three unexplained **false positives** — `/Download`, `/Password` and `/RemoteImages/Download` — all of them path fragments consistent with string concatenation, since each is the tail of a longer specification route. These are rows the tool nominated that the specification does not contain, and they should not be confused with the three specification paths the tool missed entirely, which are a different set of the same size: `/branding/css.css`, `/collections` and `/trailers`.

Interpretation, in the terms committed to above. On Jellyfin the failure is not extraction but classification. The extractor recovers very nearly everything present and its heuristic then discards about ninety per cent of the real interface while admitting a comparable volume of client routes that are not server endpoints; the heuristic is wrong in both directions at once. This is a direct quantification of the warning in Stage 6 that extracted string literals are not automatically attack surface and that the denominator must be defended rather than assumed, and it establishes that warning as a measured property of a working extractor rather than a cautionary remark. It also sharpens the argument of Section 5: if the set an assessor would treat as surface is 32.9 per cent precise and 9.5 per cent complete, then controls that make an individual negative result interpretable are not advisory, because no coverage claim computed over such a set can be trusted on its own.

Two consequences for practice follow. First, where a specification or a served sourcemap exists, classification should be performed against it rather than by a shape heuristic; this is the inverse of the usual assumption that extraction is the hard part. Second, in the absence of ground truth, the full extraction set rather than its API-shaped subset is the defensible candidate pool, since the subset was shown to exclude most of the real interface. Neither consequence affects the validity argument of Section 3.3, which concerns what a recovered specification entitles an assessor to conclude once a route is under test, not how the route was nominated.

Applying this result to the corpus figures of Section 6. The inventory figures earlier in this section were produced by the same extractor now measured, so the measurement bears on them and the arithmetic should be done here rather than left to the reader. If the client-derived portion of the corpus carries precision of the order measured above, the 1,375 routes attributed to client recovery alone correspond to a substantially smaller number of real routes — on the order of four hundred and fifty under a direct application of the 32.9 per cent figure, or five hundred and thirty under the 38.7 per cent variant. The traffic-observed portion is not subject to the same correction, because those routes were observed being requested and are real by construction.

The principal conclusion survives this correction, and survives it for a structural reason rather than a numerical one. The intersection of the two sources is 73 routes. Correcting the client-derived set downward cannot increase that intersection, and the traffic-derived set is uncorrected, so near-disjointness holds under any correction of this kind and holds more strongly the larger the correction. What the correction does affect is any reading of the raw counts as a measure of how much surface each source contributes in absolute terms, and such a reading should not be made.

Two cautions on the transfer itself. The precision figure was measured on one extractor run against one application, and the corpus inventories were additionally curated by operators rather than emitted directly by the tool, so applying the figure across targets is an approximation and not a measurement. And the corrected count is offered as an order of magnitude, not an estimate with any claim to precision of its own.

A repaired classifier, measured on three applications. The diagnosis above localises the failure to classification, which is actionable, so the classifier was rewritten and evaluated on three applications: Jellyfin, and two held out — Zulip Server 12.3 and Immich v3.2.4, each with its specification and client taken from one release tag. The keyword allowlist was replaced by call-site evidence: a path is admitted when the surrounding code passes it to an HTTP client or attaches a method to it, excluded when the surrounding code is navigation rather than request construction — a link target, a `navigate` call, a route-constant array — and otherwise retained, because discarding on absence of evidence is what produced 9.5 per cent recall. Jellyfin is the application the diagnosis came from and is not held out; Zulip and Immich are. Thresholds were fixed before measurement on each held-out application and both pre-registration files ship in the reproduction package. No pre-registration exists for Jellyfin, which bears on the tier result below, since the application where the tier separates most strongly is the one that was not held out. One criterion is applied to all three applications: **primary**, the repaired classifier's recall reaches at least 90 per cent of the measured extraction ceiling; **secondary**, precision over the set as emitted falls by no more than 5 points. A ceiling-relative primary is the correct form, because an absolute point-gain requirement would be imposed on a quantity bounded by a ceiling that differs per application.

One result generalizes, and it is the admission half. On all three applications the repaired classifier's recall equals the extraction ceiling exactly: 98.8 per cent of 243 present paths on Jellyfin, 62.2 per cent of 90 on Zulip, 82.8 per cent of 192 on Immich. The primary criterion is met at 100 per cent of the ceiling in every case. This is the same effect the Jellyfin measurement already identified and not a stronger one: the repaired rule admits essentially everything the extractor recovers, so the gain is the consequence of no longer rejecting rather than of classifying better.

The precision effect is application-dependent and can be strongly negative. Precision over the set as emitted falls 13.3 points on Zulip, from 34.0 to 20.7 per cent, and rises 12.4 points on Jellyfin, from 32.9 to 45.3 per cent; measured with absolute third-party URLs set aside the Jellyfin pair is 38.7 to 62.4 per cent. The direction tracks how many non-route rows the extractor emits on a given client: the repaired rule admits 149 absolute third-party URLs of 545 rows on Jellyfin against 11 of 73 before, and an assessor using this output needs a host filter before the set is usable. The secondary criterion is therefore met on Jellyfin and missed on Zulip. Under the single criterion the verdicts are **generalized on both axes** for Jellyfin and **partially generalized** for Zulip.

Immich is a valid test of the ceiling and an invalid test of both precision and the tier. Its web client carries no URL literals; the paths exist only inside `@immich/sdk`, which is code-generated from the same specification used here as ground truth. Every path string in the measured artefact therefore originates in the ground truth, and the resulting precision figures — 98.4 per cent for the original classifier and 98.8 for the repaired one — measure the generator rather than the classifier. **The secondary criterion is not assessable on Immich**, and its precision figures are not comparable with the other two applications. Its ceiling figure stands, because it is not circular: the specification cannot determine which paths a regex recovers. Its tier figures do not, for a reason that follows from the same circularity. A tier separates routes from false positives, and this artefact yields one false positive among 167 relative rows — the base path `/api` itself — against 177 of 424 on Jellyfin. With nothing to discriminate, every tier sits between 97.5 and 100 per cent precision and the spread is zero by construction rather than by measurement.

The evidence tier is not a general operating point. The repaired version reports an evidence tier per route. A route is assigned one of four tiers — request-construction, structural, absent-evidence, or navigation — and the spread is defined as the request-construction tier's precision minus the absent-evidence tier's, against a stated bar of 20 points for the tier to be said to carry signal. The spread is **+40.1 points on Jellyfin** (91.1 against 51.0 per cent over 112 and 251 rows) and **+9.2 points on Zulip** (28.6 against 19.4 over 28 and 62 rows). On Immich it is not assessable, for the reason given above. The spread understates the problem on Zulip, where the ordering inverts: the structural tier reaches 36.2 per cent over 177 rows, above request-construction at 28.6 over 28, so the tier is not even monotone in the strength of the evidence it records. On Jellyfin the ordering holds, request-construction at 91.1 standing above structural at 51.5 and absent-evidence at 51.0. The tier is therefore a strong precision signal on one of the two applications where it can be assessed, and on the other it is both weak and inverted. It cannot be offered as a general way to choose an operating point without calibrating it per application, which is what an operating point is supposed to spare an assessor from doing.

The extraction ceiling, and why it differs by client idiom. The ceilings are 98.8, 62.2 and 82.8 per cent, so on two of three applications the extractor fails to recover between a sixth and two fifths of the interface that is literally present in the client. Reading every unrecovered path on Immich gives two causes and no unexplained residue: 27 of 33 are template literals whose interpolation contains a nested object, as a generated query-string helper emits, and 6 carry two or more interpolations. Both reduce to one mechanism. Neither relative-path pattern can span a template interpolation whose contents are non-trivial: the character class in the plain-path pattern contains no parenthesis, so any interpolation holding a function call breaks the match, and the template pattern tolerates one interpolation of at most sixty characters containing no closing brace, which a query-string builder exceeds. Jellyfin is unaffected because its bundles carry literal `{itemId}` braces rather than interpolations, which is why its ceiling is 98.8 per cent while clients that construct paths at runtime sit lower.

A character class cannot express balanced delimiters, so widening it would admit arbitrary code into path strings. The correct fix is a scanner that consumes a quoted string while tracking interpolation nesting depth and replaces each complete interpolation with a placeholder. **It is proposed here and not applied**, so that the prediction can be recorded rather than confirmed after the fact: applying it should move Immich's ceiling from 82.8 per cent to approximately 99 per cent, every residual path being accounted for by the two causes above, and should leave Jellyfin's ceiling essentially unchanged.

Composition of the remaining false positives. Of 149 false positives among relative paths in the repaired Jellyfin run — a count that coincides with the number of absolute third-party URLs given above and denotes a disjoint set — 24 are client router paths the navigation rule did not catch and 125 are other. A minority of those are categorically not server routes: static asset paths, one unevaluated template expression, opaque fragments. Whether that noise drives the result was tested rather than assumed — excluding every string containing template syntax, whitespace, markup characters or a static-asset extension before measuring changes precision by less than one point on both runs. The residual is dominated by path fragments produced by string concatenation, some of which are prefixes of real routes; `/RemoteSearch/Subtitles/` is one, a genuine route segment the specification lists only in its full form. This is why precision against a published specification is a lower bound rather than an estimate.

Scope of this evaluation. It covers one extractor against three applications, each with one specification. The router-path class was identified after inspecting the misses rather than defined in advance, so the 88.9 per cent figure is a decomposition of the 38.7 per cent and not an independent result. The corpus figures earlier in Section 6 were produced by the original classifier and are left unchanged, because restating them under a later tool would describe a corpus that was never assessed that way. Reproduction materials accompany this manuscript in three sibling directories — `validation/` for Jellyfin, `validation-zulip/` and `validation-immich/` — each holding its specification with a digest, the output of both classifiers, and a `VALIDATION.json` listing every figure beside the version-pinned download URL and a content digest of the extracted client. The two held-out packages each additionally hold the pre-registration recording their thresholds. `validation/` holds the two comparison scripts and `classifier_rule.py`, which carries both classification rules lifted verbatim from the extractor with a self-test showing each failure direction. `reproduce_precision.py` takes a run directory, a specification and an optional base-prefix strip, and regenerates every precision, coverage and tier figure in this subsection for either classifier on any of the three applications; the tier table is computed by applying `classifier_rule.py` to the evidence fields recorded for every extracted route, so the two runs of one application yield the same table. Client archives are not identified by their own digests, because one of the three re-compresses server-side and returned different archive hashes for one pinned URL; the recorded digest is taken over the extracted file set instead. The recall script requires the client bundles, which are not redistributed; it prints the pinned URL and expected digest and exits rather than proceeding without them.

7. Threats to validity

Construct validity. The provenance and lead-type attributes operationalize their constructs through operator-assigned free-text labels, not through instrumented classification. The severity gate and adversarial review are internal criteria whose correspondence to any external standard is unestablished. Tier attribution and derived subject identity are instrument-emitted and are correspondingly stronger. The inventory construct is now partly measured rather than assumed: Section 6.2 reports the precision and completeness of the heuristic that populates it, and both are poor. It also works through what that implies for the corpus counts, including the correction it forces and the reason the non-substitutability conclusion is unaffected by it. Two bounds on that construct are weaker than a single number suggests. The completeness bound is set by extraction rather than classification and varies with client idiom — the measured ceilings are 98.8, 62.2 and 82.8 per cent across three applications — so a coverage figure computed over this inventory should be read against the ceiling of the client it came from, which for the Section 6 corpus was not measured. And the repair postdates the corpus and was not applied to it, so the construct

bounding the Section 6 figures is the original classifier, not the improved one.

Internal validity. The experience report incorporates no control condition. It does not compare the procedure against dictionary enumeration or against unassisted manual assessment, and it does not establish that equivalent outputs were unobtainable by other means. The central contribution of Section 3.3 is a validity argument, resting on the observational equivalence of a malformed request and an enforced boundary together with demonstrated instances of that failure, and a yield measurement would not strengthen it. What the report establishes is that a procedure built on the argument executes at scale against production systems and that its gate arrests a portion of its own output.

External validity. The report documents a single operator's record, produced by the procedure under study rather than by an independent instrument, across 45 targets selected by programme availability rather than by sampling design. Generalization to other operators or target populations is unsupported.

Ecological validity. The targets are production systems under authorized assessment, and the recorded responses are genuine. This constitutes the report's principal methodological strength and is the dimension on which benchmark evaluations are weakest.

Scope conditions. The argument is delimited to interface-constituted vulnerability classes and does not extend to classes requiring implementation observability: injection reaching a query constructor, unsafe deserialization, memory-safety defects, and invariant violations internal to a transaction. The structural precondition of Section 3.1 fails for applications that are not client-rendered, that construct requests server-side, or that expose their interface solely through a native client.

Sufficiency of the controls. The controls of Section 5 are necessary but not sufficient. They preclude identified invalidating conditions. They do not establish that a negative result is correct, only that it is not void for an enumerated reason.

8. Relation to prior work

Endpoint recovery from client code is established prior art, and this paper does not claim it. Sigalov and Gamayunov set out the problem in the terms used here — determining a server's HTTP endpoints is revealing its attack surface, and the client is where that surface can be inferred — and state the priority claim for inferring server endpoints from client-side JavaScript by non-trivial static analysis [5,6]. Their critique of the alternative is the same as Section 3.4's: dynamic crawling fails wherever the request-sending code cannot be triggered through interface interaction. Their observation that a single page of a single-page application frequently references the entire endpoint set is the empirical counterpart of the structural argument in Section 3.1. Wittern and colleagues treat the static analysis of API request construction in JavaScript as a software-engineering problem [7]; Gelato combines static and dynamic analysis for client-side security assessment [8]; and the industrial framing is set out in a Microsoft patent observing that REST interfaces lack a description document and that link-following scanners therefore miss endpoints and the vulnerabilities behind them [9].

What that literature supplies is the extraction technique. What it does not address is the question this paper takes up: given a recovered interface specification, what can and cannot be established with it, and under what conditions is a result measured against it valid. Three contributions follow from that question.

First, the sufficiency argument of Section 3.2 delimits the vulnerability classes for which the specification substitutes for the implementation, and concedes the classes for which it does not. The prior work establishes that endpoints can be recovered; it does not establish what recovery entitles an assessor to conclude.

Second, and principally, Section 3.3 identifies a measurement-validity consequence that the extraction literature does not treat: the recovered specification determines request well-formedness, and a malformed request is

observationally indistinguishable from an enforced authorization boundary. A procedure that recovers paths without recovering identifier encodings and body schemata will therefore generate negative results that are void rather than informative, and will not detect that this has occurred. This is a claim about the validity of measurement rather than about the yield of discovery, and it is the paper's principal contribution.

Third, the validity controls of Section 5 and the field report of Section 6 address a gap the practitioner tooling leaves open: the regex-based extractors dominating practice have no published precision or recall evaluation. Section 6.2 supplies one for the extractor used here, against a published specification as ground truth, and the result is unflattering in a specific and informative way — on Jellyfin, near-complete extraction paired with a classification heuristic that is 32.9 per cent precise and 9.5 per cent complete. Reporting it is the point: a paper arguing that measurement validity is the neglected problem in this area cannot leave its own measuring instrument unmeasured. The diagnosis was also actionable, and acting on it is what bounds the claim: replacing the keyword heuristic with call-site evidence and measuring both rules on three applications shows one effect that generalizes, recovery of the full extraction ceiling, and two that do not, precision and the evidence tier. The three applications also relocate the larger loss from classification to extraction on the two clients that build paths at runtime. Both rules and all six runs ship with this manuscript so that each claim can be checked rather than believed.

Separately, work on language-model agents demonstrates exploitation of known one-day vulnerabilities [1] and multi-agent operation against zero-day targets [2], in both cases with access to the code under assessment. These motivate the opening question of Section 1 — what substitutes for readable code when the server is not disclosed — rather than bearing on the answer.

9. Conclusion

AI-assisted assessment is effective against source code because the implementation is observable. Web applications do not expose their implementation, but they do expose their interface specification, since the client invoking that interface is distributed in its entirety to every user agent and must encode whatever a well-formed request requires. Source inspection and client recovery therefore answer distinct questions, and no equivalence is claimed.

The claim advanced is that the question client recovery answers is the one determinative for the vulnerability classes the standard interface taxonomy ranks highest. Broken object level authorization (API1:2023, CWE-639), broken object property level authorization (API3:2023, CWE-915), broken function level authorization (API5:2023), sensitive business flow access (API6:2023), and behavioural workflow enforcement failure (CWE-841) are determined by what a server accepts from a caller. For these the interface specification is sufficient, the confirming evidence is a request together with a subject contrast, and the specification recovered from the invoking code is what renders the request well-formed — which is equally what renders a negative result informative. Neither recovery nor validity is the easy half. Section 6.2 measures extraction ceilings of 98.8, 62.2 and 82.8 per cent across three clients, so recovery is itself incomplete wherever a client builds its paths at runtime, and the validity controls are what prevent the volume that is recovered from constituting high-confidence false positives. The clearest internal evidence of their operation is the record of candidates the procedure declined to advance.

References

- [1] Fang, R., Bindu, R., Gupta, A., and Kang, D. (2024). *LLM Agents can Autonomously Exploit One-day Vulnerabilities*. arXiv:2404.08144v2. <https://arxiv.org/abs/2404.08144v2>
- [2] Zhu, Y., et al. (2025 revision). *Teams of LLM Agents can Exploit Zero-Day Vulnerabilities*. arXiv:2406.01637v2. <https://arxiv.org/abs/2406.01637v2>

- [3] OWASP Foundation. *OWASP API Security Top 10* — 2023. <https://api-security.owasp.org/editions/2023/en/0x11-t10> Accessed 1 October 2026.
- [4] MITRE. *Common Weakness Enumeration*. CWE-639, *Authorization Bypass Through User-Controlled Key*; CWE-915, *Improperly Controlled Modification of Dynamically-Determined Object Attributes*; CWE-841, *Improper Enforcement of Behavioral Workflow*; CWE-602, *Client-Side Enforcement of Server-Side Security*. <https://cwe.mitre.org/> Accessed 1 October 2026.
- [5] Sigalov, D., and Gamayunov, D. (2024). *Finding Server-Side Endpoints with Static Analysis of Client-Side JavaScript*. In *Computer Security — ESORICS 2023 International Workshops*. Lecture Notes in Computer Science, vol. 14399, pp. 442–458. Springer, Cham. https://doi.org/10.1007/978-3-031-54129-2_26
- [6] Sigalov, D., and Gamayunov, D. (2024). *Dead or alive: Discovering server HTTP endpoints in both reachable and dead client-side code*. *Journal of Information Security and Applications* 82, article 103746.
- [7] Wittern, E., Ying, A. T. T., Zheng, Y., Dolby, J., and Laredo, J. A. (2017). *Statically Checking Web API Requests in JavaScript*. In *Proceedings of the 39th International Conference on Software Engineering (ICSE)*.
- [8] Hassanshahi, B., Lee, H., and Krishnan, P. (2022). *Gelato: Feedback-driven and Guided Security Analysis of Client-side Web Applications*. In *IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. arXiv:2004.06292. <https://arxiv.org/abs/2004.06292>
- [9] Microsoft Technology Licensing, LLC. *Extracting web API endpoint data from source code to identify potential security threats*. United States Patent 10,754,628.